Faculty of Engineering and Technology
Object Oriented Programming (CEUE203)
A.Y 2026 – 27 Semester – 3
Practical List

Practical Number		CO/PO
11	**Part A — Practice Programs (Hour 1)** Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. 1. Save/load: make a small class Serializable, save an array of its objects to a file with ObjectOutputStream, then read it back into fresh objects and confirm the values survived. Mark one field transient and show it is not saved. 2. Log analyzer: create a few text files, then use Path/Files to read each line by line and count the total lines and how many contain a keyword across all files; also print each file's size from its attributes. 3. (For Advanced Learners – home) Walk a folder tree with NIO and produce a report of the files it contains (name, size, modified time), writing the summary to a file; decide what to include. **Part B — Project Work (Hour 2): MiniBank** So far MiniBank forgets everything when it closes. In this lab you add the persistence and reporting layer: you make the model Serializable and save/load all accounts to a file, write an append-only transaction log, and then use NIO to walk the logs and produce an end-of-day summary report. 1. Make Account (and its subclasses and Customer) implement Serializable, and add a private static final long serialVersionUID field. 2. Create a class named StatePersister with a public static method void save(Account[] accounts, Path file) that uses ObjectOutputStream inside try-with-resources, and a public static method Account[] load(Path file) that uses ObjectInputStream. 3. Create a class named TransactionLog with a public static method void append(Path file, String line) that uses Files.write(...) with CREATE and APPEND. 4. Create a class named ReportGenerator that walks the logs folder using Files.newDirectoryStream, reads each file line by line with a BufferedReader, and adds up the total deposits and total withdrawals in long variables; also read file details (size, last-modified) using Files.readAttributes, then write the summary to a report file. 5. In main, save the accounts, reload them into a new array and print them to confirm the data survived, then generate and print the report. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. What does it mean for a class to be Serializable, and why add serialVersionUID? 2. What is the difference between a byte stream and a character stream? 3. How does NIO's Path/Files API differ from the old File class, and how is a directory stream iterated? **Supplementary Problems –** • Mark one field transient and show it is not saved. • Skip files that are empty or do not end with .log when generating the report. **Key Skills to be addressed –** Byte and character streams; object serialization and deserialization; Path and Files; try-with-resources; directory streams; file attributes; producing a report file. **Applications –** Saving application state, audit logging and end-of-day reporting are required in almost every program. **Learning Outcome –** The student can persist objects to files, reload them, and summarise log data using NIO. (CO5)	**CO5** PO4, PO5, PSO1, PSO2
	Dataset / Test Data (Source and Description, if applicable): The program creates a data folder at run time; the saved file (accounts.dat) and the log files are produced by the program itself and become the input to the report.	

Practical Number		CO/PO
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	
	Post Laboratory Work Description: Commit and tag lab-11 (do not commit the generated data files). Confirm that data saved in one run is loaded correctly in the next, and that the report totals match the logged transactions.	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Serializable model and save/load – 35%; transaction log – 15%; NIO report with correct totals – 30%; naming conventions and viva – 20%.	
	Expected Input / Process / Output **Input:** In code: an Account[] of three accounts saved to accounts.dat then reloaded; a logs folder with lines such as "DEPOSIT AC0001 500" and "WITHDRAW AC0002 200". **Process:** save() writes the array with ObjectOutputStream and load() reads it back; the report reads every log line and adds amounts to the deposit or withdrawal total based on the first word. **Expected Output:** The reloaded accounts print with the same numbers and balances as the saved ones; the report file shows the total deposits, total withdrawals and the net change.	